Object Oriented Programming (CSEG1044)

Unit 01 – Lecture 03: I/O + Command Line + Types + Operators

April 1, 2026

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

Quick Links

Overview Key Topics Demo Summary

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

Agenda

1 Overview

2 Key Topics

3 Demo / Example

4 Summary

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

Lecture Snapshot

- **Unit focus:** Introduction to OOPs and Java Fundamentals

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

Lecture Snapshot

- **Unit focus:** Introduction to OOPs and Java Fundamentals
- **Course thread:** Use I/O + Command Line + Types + Operators to make the first text-based version of Campus Resource Manager easier to reason about before the full object model arrives.

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

Recall Before You Start

- **Program structure**: You already know where execution starts in Java; this lecture fills that structure with input and expressions. Main reference: `Unit 01 / Lecture 02 slides/notes`.

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

Recall Before You Start

- **Program structure**: You already know where execution starts in Java; this lecture fills that structure with input and expressions. Main reference: `Unit 01 / Lecture 02 slides/notes`.
- **Basic data handling**: The C prerequisite gives the intuition for values and operators; Java adds stronger typing rules. Main reference: Programming in C prerequisite.

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

Learning Outcomes

- Read input using `Scanner` and print formatted output.

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

Learning Outcomes

- Read input using `Scanner` and print formatted output.
- Use command line arguments (`args`) for simple programs.

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

Learning Outcomes

- Read input using `Scanner` and print formatted output.
- Use command line arguments (`args`) for simple programs.
- Apply primitive types, operators, precedence, and casting correctly.

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

Key Terms to Know

- **Token vs line**: token = one word/number; line = full line of text.

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

Key Terms to Know

- **Token vs line**: token = one word/number; line = full line of text.
- **Formatting**: printing numbers/text with a chosen format (decimal places, alignment).

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

Key Terms to Know

- **Token vs line**: token = one word/number; line = full line of text.
- **Formatting**: printing numbers/text with a chosen format (decimal places, alignment).
- **Primitive types**: built-in value types like `int`, `double`, `char`, `boolean`.

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

Key Terms to Know

- **Token vs line**: token = one word/number; line = full line of text.
- **Formatting**: printing numbers/text with a chosen format (decimal places, alignment).
- **Primitive types**: built-in value types like `int`, `double`, `char`, `boolean`.
- **Type casting**: converting a value from one type to another (widening/narrowing).

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

Key Terms to Know

- **Token vs line**: token = one word/number; line = full line of text.
- **Formatting**: printing numbers/text with a chosen format (decimal places, alignment).
- **Primitive types**: built-in value types like `int`, `double`, `char`, `boolean`.
- **Type casting**: converting a value from one type to another (widening/narrowing).
- **Precedence**: order in which operators are evaluated.

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

Study Flow for This Lecture

1. Refresh the prerequisite bridge before reading new ideas in isolation.

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

Study Flow for This Lecture

1. Refresh the prerequisite bridge before reading new ideas in isolation.
2. Study the main build in this order: *Console I/O: `Scanner` (most common for beginners)* → *Console I/O: `BufferedReader` (fast, line-oriented)* → *Output formatting* → *Command line arguments*.

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

Study Flow for This Lecture

1. Refresh the prerequisite bridge before reading new ideas in isolation.
2. Study the main build in this order: *Console I/O: `Scanner` (most common for beginners)* → *Console I/O: `BufferedReader` (fast, line-oriented)* → *Output formatting* → *Command line arguments*.
3. Trace the worked example and connect each line back to one concept frame.

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

Study Flow for This Lecture

1. Refresh the prerequisite bridge before reading new ideas in isolation.
2. Study the main build in this order: *Console I/O: `Scanner` (most common for beginners)* → *Console I/O: `BufferedReader` (fast, line-oriented)* → *Output formatting* → *Command line arguments*.
3. Trace the worked example and connect each line back to one concept frame.
4. Attempt the mini exercises before moving to the recap and exit check.

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

Unit 01: Introduction to OOPs

- Lecture focus: I/O + Command Line + Types + Operators

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

Today: Roadmap

- Console I/O (‘Scanner‘, ‘BufferedReader‘) and output formatting

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

Today: Roadmap

- Console I/O (‘Scanner‘, ‘BufferedReader‘) and output formatting
- Command line arguments and use-cases

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

Today: Roadmap

- Console I/O (‘Scanner‘, ‘BufferedReader‘) and output formatting
- Command line arguments and use-cases
- Data types, variables, operators, precedence, type casting

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

Console input: `Scanner` (beginner-friendly)

- Reads **tokens** from `System.in`: `nextInt()`, `nextDouble()`, `next()`.

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

Console input: `Scanner` (beginner-friendly)

- Reads **tokens** from `System.in`: `nextInt()`, `nextDouble()`, `next()`.
- Reads **lines**: `nextLine()`.

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

Console input: `Scanner` (beginner-friendly)

- Reads **tokens** from `System.in`: `nextInt()`, `nextDouble()`, `next()`.
- Reads **lines**: `nextLine()`.
- Common pitfall: mixing `nextInt()` then `nextLine()` (newline remains in buffer).

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

Console input: `Scanner` (beginner-friendly)

- Reads **tokens** from `System.in`: `nextInt()`, `nextDouble()`, `next()`.
- Reads **lines**: `nextLine()`.
- Common pitfall: mixing `nextInt()` then `nextLine()` (newline remains in buffer).
- Fix: consume leftover newline with one extra `nextLine()`.

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

Output formatting: `printf`

- `println` is fine for simple output.

Examples

- `%d` integer, `%f` decimal, `%.2f` 2 decimals, `%s` string
- `%n` portable newline

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

Output formatting: `printf`

- `println` is fine for simple output.
- `printf` is better for formatted numbers/text.

Examples

- `%d` integer, `%f` decimal, `%.2f` 2 decimals, `%s` string
- `%n` portable newline

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

Command line arguments (`args`)

- Run: `java Sum 10 20 30`

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

Command line arguments (`args`)

- Run: `java Sum 10 20 30`
- Then: `args[0] = "10"`, `args[1] = "20"`, ...

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

Command line arguments (`args`)

- Run: `java Sum 10 20 30`
- Then: `args[0] = "10"`, `args[1] = "20"`, ...
- Parse numbers: `Integer.parseInt(args[i])`

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

Command line arguments (`args`)

- Run: `java Sum 10 20 30`
- Then: `args[0] = "10"`, `args[1] = "20"`, ...
- Parse numbers: `Integer.parseInt(args[i])`
- Pitfall: invalid input throws `NumberFormatException`

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

Primitive types (quick)

- Integers: `byte`, `short`, `int`, `long`

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

Primitive types (quick)

- Integers: `byte`, `short`, `int`, `long`
- Floating-point: `float`, `double`

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

Primitive types (quick)

- Integers: `byte`, `short`, `int`, `long`
- Floating-point: `float`, `double`
- Others: `char` (Unicode), `boolean`

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

Operator pitfalls (must know)

- Integer division: `5/2 == 2` (use `5/2.0`)

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

Operator pitfalls (must know)

- Integer division: `5/2 == 2` (use `5/2.0`)
- Precedence: multiplication before addition (use parentheses when unsure)

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

Operator pitfalls (must know)

- Integer division: `5/2 == 2` (use `5/2.0`)
- Precedence: multiplication before addition (use parentheses when unsure)
- Casting: narrowing needs explicit cast (may lose data)

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

Operator pitfalls (must know)

- Integer division: `5/2 == 2` (use `5/2.0`)
- Precedence: multiplication before addition (use parentheses when unsure)
- Casting: narrowing needs explicit cast (may lose data)
- Short-circuit: `&&` and `||` help safe null checks

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

Checkpoint and Reflection

Reflection prompt

Where would *I/O + Command Line + Types + Operators* matter most in Campus Resource Manager, and which design choice would you justify using this lecture's ideas?

Quick self-check

Which part needs one more example before you move on: *Console I/O: `Scanner` (most common for beginners)*, *Console I/O: `BufferedReader` (fast, line-oriented)*, the worked example, or the recap?

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

Example: Args + Scanner + Formatting (1/2)

```
import java.util.Scanner;

public class InputDemo {
    public static void main(String[] args) {
        int maxMarks = 100;
        if (args.length > 0) maxMarks = Integer.parseInt(
            args[0]);

        Scanner sc = new Scanner(System.in);
        System.out.print("Enter your name: ");
        String name = sc.nextLine();
```

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

Example: Args + Scanner + Formatting (2/2)

```
// continued...
        System.out.print("Enter marks (0-" + maxMarks + "):
            ");
        int marks = sc.nextInt();

        double percent = marks * 100.0 / maxMarks;
        System.out.printf("Hello %s, marks=%d/%d, percent
            =%.2f%%%n",
                name, marks, maxMarks, percent);
        sc.close();
    }
}
```

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

Mini Exercises (2–3 mins)

1. Predict output: `System.out.println(5/2);`

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

Mini Exercises (2–3 mins)

1. Predict output: `System.out.println(5/2);`
2. How to get 2.5 from 5 and 2?

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

Mini Exercises (2–3 mins)

1. Predict output: `System.out.println(5/2);`
2. How to get 2.5 from 5 and 2?
3. If you run `java Demo A B`, what is `args.length`?

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

Watch-outs

- Mixing numeric and line-based input without understanding Scanner behavior.

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

Watch-outs

- Mixing numeric and line-based input without understanding Scanner behavior.
- Ignoring operator precedence and misreading the result.

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

Watch-outs

- Mixing numeric and line-based input without understanding Scanner behavior.
- Ignoring operator precedence and misreading the result.
- Expecting Java to convert unrelated types automatically.

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

Key Takeaways

- `Scanner` is easy for input; know the `nextInt()` vs `nextLine()` pitfall.

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

Key Takeaways

- `Scanner` is easy for input; know the `nextInt()` vs `nextLine()` pitfall.
- `printf` makes output readable (format specifiers).

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

Key Takeaways

- `Scanner` is easy for input; know the `nextInt()` vs `nextLine()` pitfall.
- `printf` makes output readable (format specifiers).
- Command line args are strings; parse to numbers when needed.

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

Key Takeaways

- `Scanner` is easy for input; know the `nextInt()` vs `nextLine()` pitfall.
- `printf` makes output readable (format specifiers).
- Command line args are strings; parse to numbers when needed.
- Watch integer division, precedence, and casting.

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

Checkpoint Questions

1. Why can `nextLine()` return empty after `nextInt()`?

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

Checkpoint Questions

1. Why can `nextLine()` return empty after `nextInt()`?
2. What is the type of `args[i]`?

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

Checkpoint Questions

1. Why can `nextLine()` return empty after `nextInt()`?
2. What is the type of `args[i]`?
3. How do you avoid integer division when you want decimals?

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

Next Study Steps

- Revisit the recall references if any older idea still feels weak.

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

Next Study Steps

- Revisit the recall references if any older idea still feels weak.
- Rework the lecture example without looking at the notes first.

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

Next Study Steps

- Revisit the recall references if any older idea still feels weak.
- Rework the lecture example without looking at the notes first.
- Attempt the self-study practice and then answer the exit questions in full sentences.

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

Next Study Steps

- Revisit the recall references if any older idea still feels weak.
- Rework the lecture example without looking at the notes first.
- Attempt the self-study practice and then answer the exit questions in full sentences.
- Apply the idea once inside Campus Resource Manager to make the concept stick.

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

Next Lecture Preview

Next: Control Flow + Arrays + Strings

- Bring one example where you used a loop in C.

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).

Next Lecture Preview

Next: Control Flow + Arrays + Strings

- Bring one example where you used a loop in C.
- We will solve simple array + string problems together.

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed April 1, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed April 1, 2026).